

For the task 1 just run the tcp-chat file and provide 0/port number as arguments to start server or ip&port number to connect with client.

T1: 3/3  
T2: 3/3  
T3: 3/3  
T4: 1/1  
Sum: 10/10

Next time please be ready for interview.

## Task 1 Screenshots & Report: .pcapng not in zip

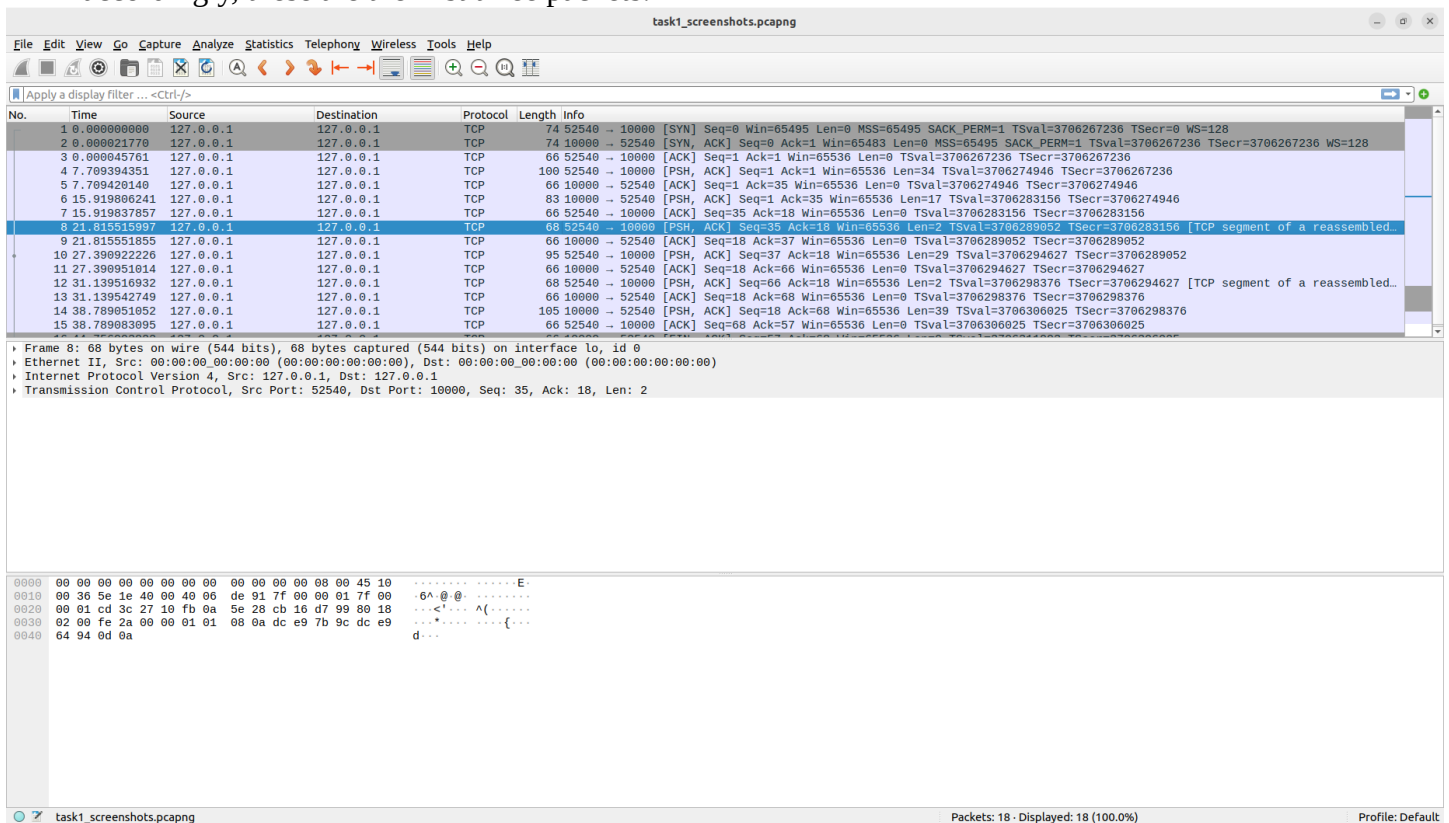
The network protocol that is used for all the packets is TCP.

### 1. When the connection is first established

#### a. Packets & contents

In the beginning you can see the initial handshake between the client and the server. This involves a three-way handshake where the client sends a SYN packet, the server responds with a SYN-ACK, and finally, the client sends an ACK. → You can see which is sending which from the port numbers of the source bzw. destination ports. (Source and destination IP is localhost).

In Wireshark you can see the TCP packets with the SYN, SYN-ACK and ACK flags set accordingly, these are the first three packets:

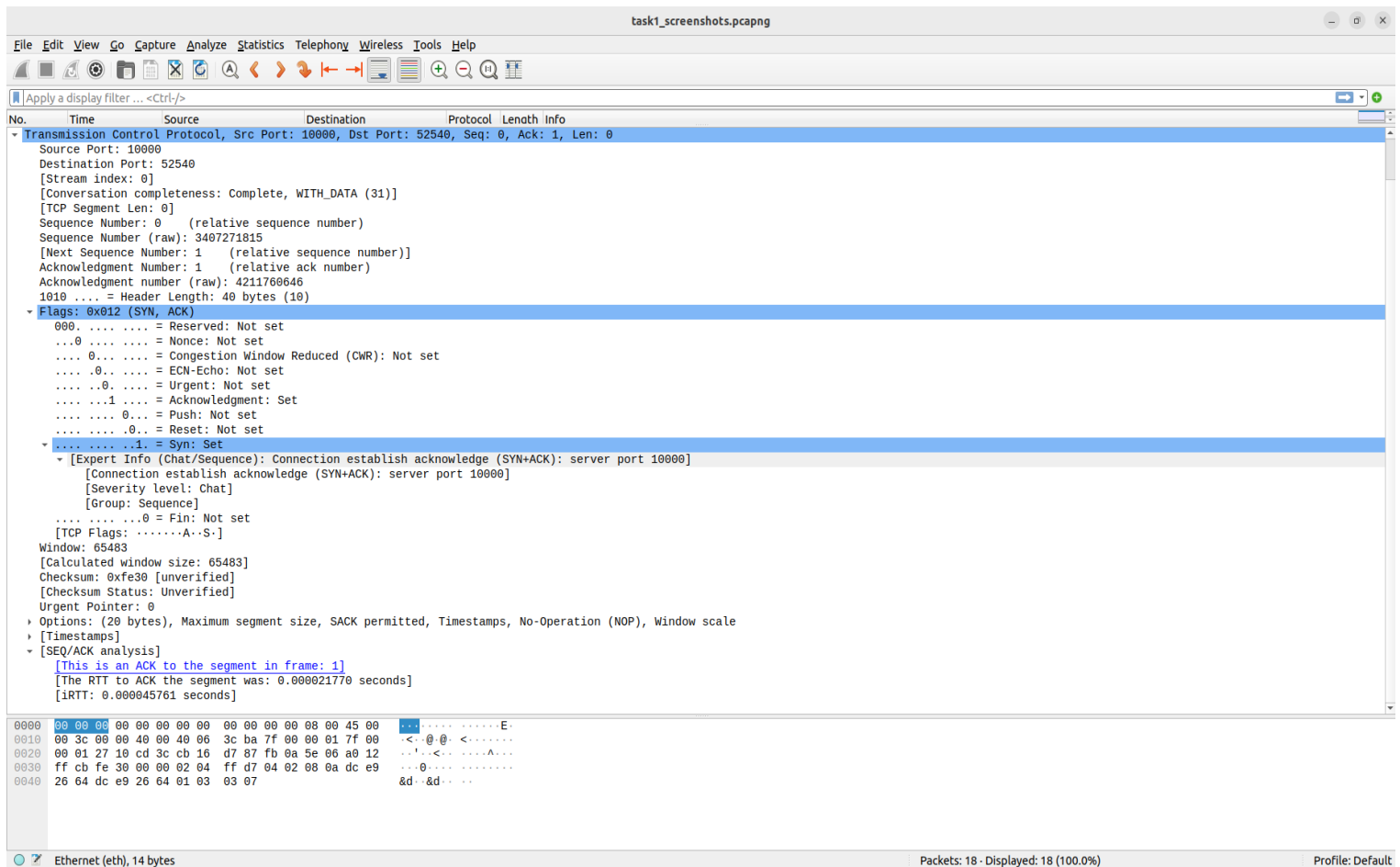


The picture is also in the .zip, I had a hard time making it readable size here.



#### b. Contents

In the contents pane of the relevant packet you can see for example which IP version is used or if the checksum of the header is calculated. You can also see the length of the frame of the packet. You can also see which flags have been set or the acknowledgment number which is used in TCP.



## 2. When messages are exchanged

### a. Packets & contents

When analyzing the packets exchanged when messages are sent between the client and the server you can see the TCP packets containing the actual text of the messages, you can see this in the screenshots provided (but better in the actual .pcapng Wireshark file in the .zip folder).

The TCP packets containing the text of the messages are easily identifiable by their payload, which contains the text entered during the chat.

Since Wireshark allows you to view the payload of TCP packets, it will show you the actual content of the messages exchanged.

On the bottom of the picture you can see the first message:



### a. Packets & contents

When the connection is terminated you can see the involved TCP FIN packet sent by, in my case, the server (flag: FIN, ACK) to indicate the intention to close the connection, followed by and acknowledgment (flag: FIN, ACK) from the other party (the terminal using telnet). Finally the server sends the last acknowledgment (flag: ACK).

In Wireshark you can see on the bottom the last three packets with the fags and ports:

**task1\_screenshots.pcapng**

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

| No. | Time         | Source    | Destination | Protocol | Length | Info                                                                                                                        |
|-----|--------------|-----------|-------------|----------|--------|-----------------------------------------------------------------------------------------------------------------------------|
| 1   | 0.000000000  | 127.0.0.1 | 127.0.0.1   | TCP      | 74     | 52540 → 10000 [SYN] Seq=0 Win=5495 Len=0 MSS=65495 SACK_PERM=1 TSval=3706267236 TSecr=0 WS=128                              |
| 2   | 0.000021770  | 127.0.0.1 | 127.0.0.1   | TCP      | 74     | 10000 → 52540 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=3706267236 TSecr=3706267236 WS=128         |
| 3   | 0.000045761  | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 52540 → 10000 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3706267236 TSecr=3706267236                                           |
| 4   | 7.709394351  | 127.0.0.1 | 127.0.0.1   | TCP      | 100    | 52540 → 10000 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=34 TSval=3706274946 TSecr=3706267236                                     |
| 5   | 7.709420148  | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 10000 → 52540 [ACK] Seq=1 Ack=35 Win=65536 Len=0 TSval=3706274946 TSecr=3706274946                                          |
| 6   | 15.919806241 | 127.0.0.1 | 127.0.0.1   | TCP      | 83     | 10000 → 52540 [PSH, ACK] Seq=1 Ack=35 Win=65536 Len=17 TSval=3706283156 TSecr=3706274946                                    |
| 7   | 15.919837857 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 52540 → 10000 [ACK] Seq=35 Ack=18 Win=65536 Len=0 TSval=3706283156 TSecr=3706283156                                         |
| 8   | 21.815515997 | 127.0.0.1 | 127.0.0.1   | TCP      | 68     | 52540 → 10000 [PSH, ACK] Seq=35 Ack=18 Win=65536 Len=2 TSval=3706289052 TSecr=3706283156 [TCP segment of a reassembled PDU] |
| 9   | 21.815515855 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 10000 → 52540 [ACK] Seq=18 Ack=37 Win=65536 Len=0 TSval=3706289052 TSecr=3706289052                                         |
| 10  | 27.390922226 | 127.0.0.1 | 127.0.0.1   | TCP      | 95     | 52540 → 10000 [PSH, ACK] Seq=37 Ack=18 Win=65536 Len=29 TSval=3706294627 TSecr=3706289052                                   |
| 11  | 27.390951014 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 10000 → 52540 [ACK] Seq=18 Ack=66 Win=65536 Len=0 TSval=3706294627 TSecr=3706294627                                         |
| 12  | 31.139516932 | 127.0.0.1 | 127.0.0.1   | TCP      | 68     | 52540 → 10000 [PSH, ACK] Seq=66 Ack=18 Win=65536 Len=2 TSval=3706298376 TSecr=3706294627 [TCP segment of a reassembled PDU] |
| 13  | 31.139542749 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 10000 → 52540 [ACK] Seq=18 Ack=68 Win=65536 Len=0 TSval=3706298376 TSecr=3706298376                                         |
| 14  | 30.709051052 | 127.0.0.1 | 127.0.0.1   | TCP      | 105    | 10000 → 52540 [PSH, ACK] Seq=18 Ack=68 Win=65536 Len=39 TSval=3706306025 TSecr=3706298376                                   |
| 15  | 38.709083995 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 52540 → 10000 [ACK] Seq=68 Ack=57 Win=65536 Len=0 TSval=3706306025 TSecr=3706306025                                         |
| 16  | 44.756993225 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 10000 → 52540 [FIN, ACK] Seq=57 Ack=68 Win=65536 Len=0 TSval=3706311993 TSecr=3706306025                                    |
| 17  | 44.757086645 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 52540 → 10000 [FIN, ACK] Seq=68 Ack=58 Win=65536 Len=0 TSval=3706311993 TSecr=3706311993                                    |
| 18  | 44.757101996 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 10000 → 52540 [ACK] Seq=58 Ack=69 Win=65536 Len=0 TSval=3706311993 TSecr=3706311993                                         |

Frame 6: 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface lo, id 0  
Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)  
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1  
Transmission Control Protocol, Src Port: 10000, Dst Port: 52540, Seq: 1, Ack: 35, Len: 17  
Data (17 bytes)

```

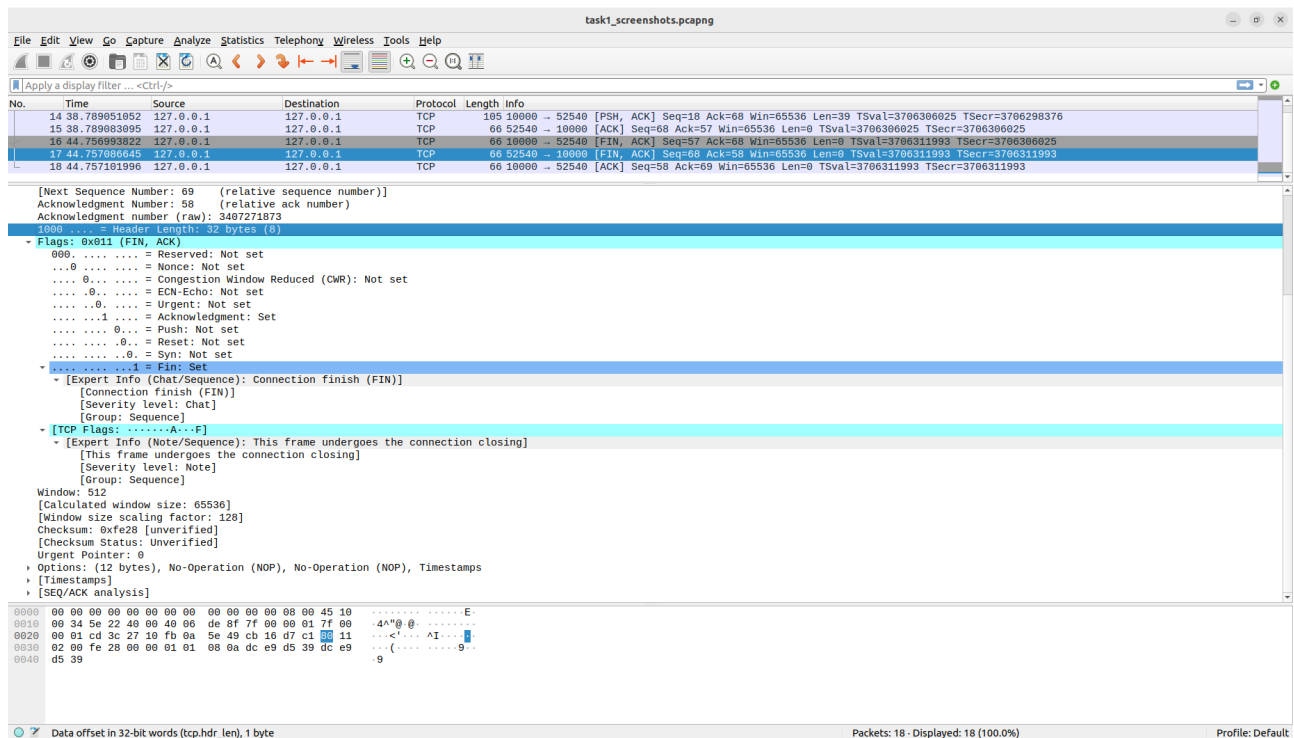
0000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 45 00  E.....E..
0018 00 45 df c0 40 00 40 06 5c f0 7f 00 00 01 7f 00  E... ..
0020 00 01 27 10 cd 3c cb 16 d7 88 fb 0a 5e 28 00 18  ....<. . . . ^(.
0030 02 00 fe 39 00 00 01 01 08 0a dc e9 64 94 dc e9  .9.....d....
0040 44 82 68 05 6c 6c 6f 29 73 65 72 76 65 72 20 68  D.hello server h
0050 65 72 05                                     ere

```

Ethernet (eth), 14 bytes

Packets: 18 · Displayed: 18 (100.0%) Profile: Default

Regarding the contents you could also look at the set TCP flags in the content pane:



Chat works

## Task 2:

Once you run the script for the first time you have to start a rendez-vous server by typing server. If you start a client by typing client after running the script you have to give the name of the rendez-vous server and choose a nickname.

In my implementation both clients have to request each others nickname from the rendez-vous server in order to chat with each other.

Ok works, name of server is apparently ip (or localhost).

## Task 3:

Works

Just run the file and choose a nickname and you will receive all the messages from everybody else that has run the file and chose a nickname as well as being able to send messages to all these people too.

## Task 3 Question:

To implement a feature for displaying a current list of all the other participants in the network in this decentralized chat the following would be implementation options:

**Participants Finding Protocol:** Implement a protocol where clients periodically broadcast a "hello" message containing their nickname to the broadcast address. Other clients listen for these messages and maintain a list of active participants themselves.

**Heartbeat Messages:** In order to keep the list up to date apart from explicitly leaving clients, clients can periodically broadcast "heartbeat" messages to indicate their presence in the network. If a client stops receiving heartbeat messages from a participant for a certain duration, it assumes that the participant has left the network or disconnected.

**Joining Clients:** When a new client joins the network, it broadcasts a "hello" message (compliant with the protocol that one made up). Existing clients receive this message and add the new participant to their list.

**Leaving Clients:** When a client wishes to leave the network, it broadcasts a "goodbye" message. Other clients receiving this message remove the departing participant from their list.

**Participant List Exchange:** Optionally, one could implement a mechanism where clients periodically exchange their participant lists. This can help in case a client misses some "hello" or "goodbye" messages due to network issues. ✓✓✓

These functionalities would be implemented at the application layer of the network model (the highest layer). Clients would manage the participant list locally, and communication regarding joining/leaving participants would occur directly between clients through broadcast messages.

Nice

Potential downsides of a decentralized model:

**Network Congestion:** Frequent broadcasting of messages for participant discovery and heartbeats could lead to network congestion, especially in larger networks.

**Scalability:** As the number of participants increases, managing the participant list and handling frequent broadcasts could become more complex and resource-intensive.

**Reliability:** Broadcasting messages may not always guarantee delivery, especially in large or congested networks. Some participants might miss messages, leading to inconsistencies in the participant list. Resolving these inconsistencies I imagine to be harder than with a server.

**Security:** Without a central server, it may be more challenging to implement security measures such as authentication and encryption. For example one could log out another user with the leaving protocol and the log in as that user, stopping that I think may be harder in a decentralized than in a centralized model.

Well thought out!

#### Task 4 Question:

The main difference in the content of relevant packets from task 1 compared to accessing an HTTPS website (HTTPS/TCP) would be that the contents of the packets are encrypted. Apparently, HTTPS/TCP communication involves encryption using SSL/TLS, making the payload of the packets unreadable without decryption keys.

✓